

Laboratorio No. 8

Juan Menéndez

7 de octubre de 2026

Ejercicio 1

a) Complejidad de tiempo en notación Big-Oh

El algoritmo se compone de tres ciclos anidados:

1. **Ciclo exterior (i):** Inicia en $n/2$ y termina en n . El número de iteraciones es aproximadamente $n - n/2 = n/2$, lo cual representa una complejidad lineal $O(n)$.
2. **Ciclo intermedio (j):** La condición es $j + n/2 \leq n$, equivalente a $j \leq n/2$. Itera desde 1 hasta $n/2$, representando también una complejidad $O(n)$.
3. **Ciclo interior (k):** Inicia en 1 y se duplica en cada iteración ($k = k \times 2$) hasta alcanzar n . El número de iteraciones es $\log_2(n)$, resultando en $O(\log n)$.

Al estar anidados, las complejidades se multiplican: $O(n) \times O(n) \times O(\log n) = O(n^2 \log n)$. La complejidad de tiempo final es $O(n^2 \log n)$.

b) Profiling y Resultados

Tamaño (n)	Tiempo de Ejecución
1	711 ns
10	652 ns
100	48.038 μs
1000	6.217346 ms
10000	497.096717 ms
100000	53.376265852 s

Cuadro 1: Tiempos de ejecución obtenidos con Go para el Ejercicio 1.

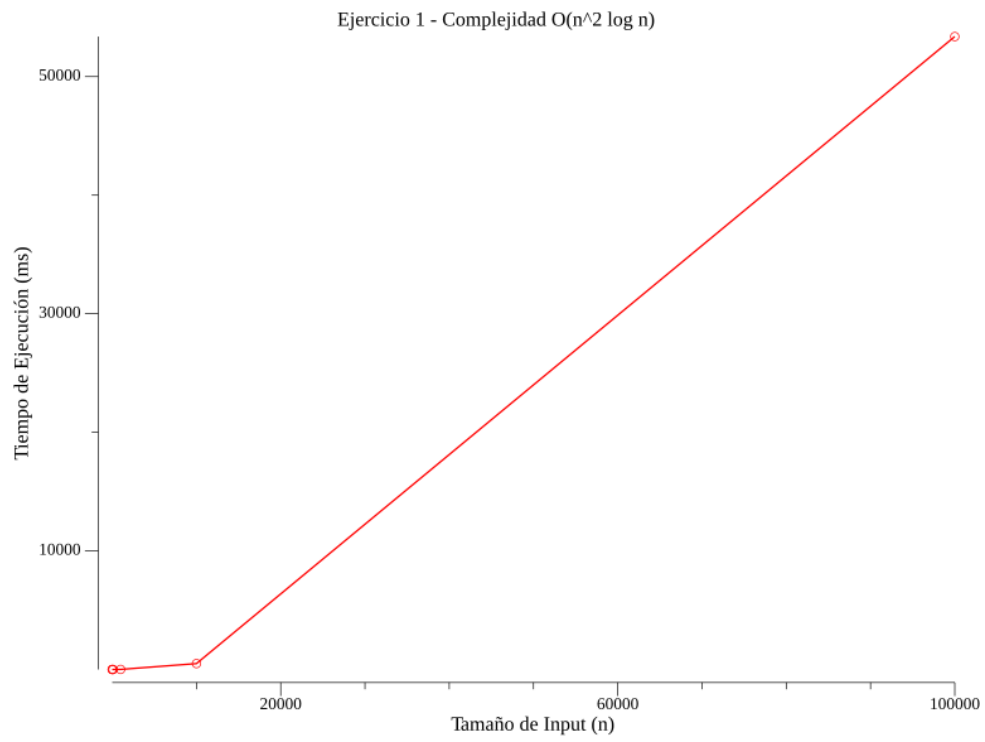


Figura 1: Gráfica de crecimiento $O(n^2 \log n)$.

Ejercicio 2

a) Complejidad de tiempo en notación Big-Oh

A primera vista existen dos ciclos anidados, pero se debe notar la ruptura estructural en el flujo de control:

1. **Ciclo exterior (i):** Se ejecuta linealmente desde 1 hasta n .
2. **Ciclo interior (j):** Aunque la declaración indica que va de 1 a n , el bloque contiene una instrucción **break** incondicional en su primera línea. Esto significa que el ciclo se interrumpe inmediatamente en su primera iteración ($j = 1$), sin importar el valor de n .

El ciclo interior ejecuta sus instrucciones exactamente 1 vez por cada iteración del ciclo exterior. El costo total es $n \times 1 = n$. La complejidad temporal es estrictamente lineal $O(n)$.

b) Profiling y Resultados

Tamaño (n)	Tiempo de Ejecución
1	581 ns
10	320 ns
100	381 ns
1000	1.353 μs
10000	11.141 μs
100000	71.243 μs
1000000	716.808 μs

Cuadro 2: Tiempos de ejecución obtenidos con Go para el Ejercicio 2.

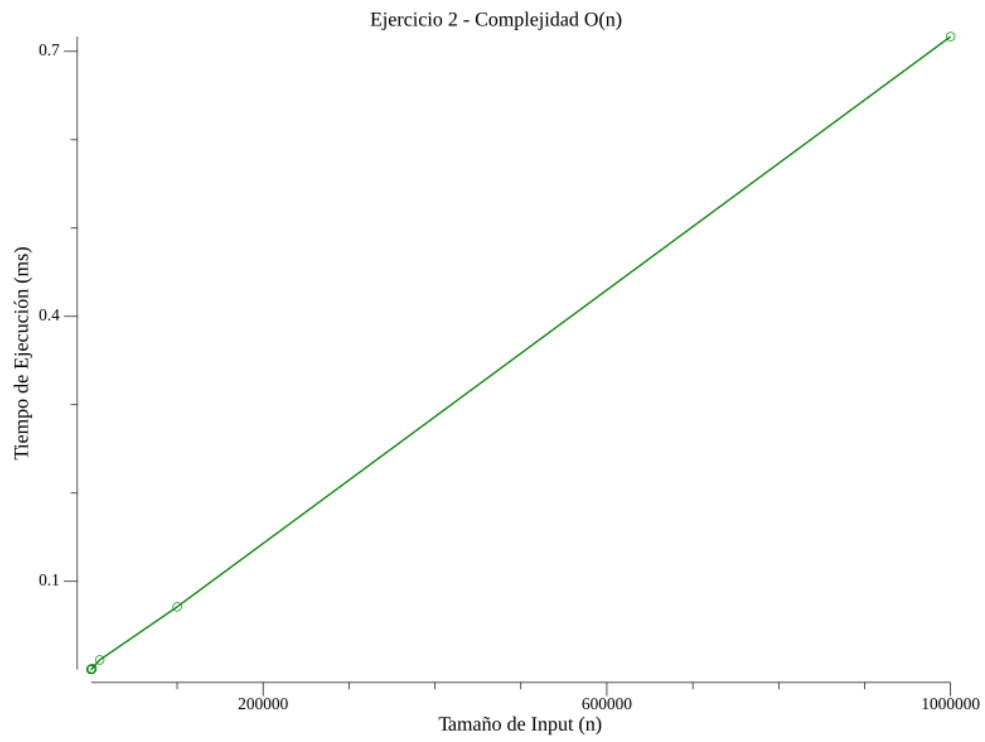


Figura 2: Gráfica de crecimiento lineal $O(n)$.

Ejercicio 3

a) Complejidad de tiempo en notación Big-Oh

Análisis de los ciclos de la función:

1. **Ciclo exterior (i):** Inicia en 1 y avanza hasta $n/3$ con saltos unitarios. El número de iteraciones es $n/3$.
2. **Ciclo interior (j):** Inicia en 1 y avanza hasta n con saltos de 4 unidades por iteración ($j += 4$). El número de iteraciones es $n/4$.

Al estar anidados, multiplicamos ambas iteraciones: $(n/3) \times (n/4) = n^2/12$. En notación asintótica se descartan las constantes multiplicativas o divisoras, dejando el término dominante. Por lo tanto, la complejidad de tiempo es $O(n^2)$.

b) Profiling y Resultados

Tamaño (n)	Tiempo de Ejecución
1	571 ns
10	291 ns
100	932 ns
1000	67.978 μs
10000	5.975296 ms
100000	432.299803 ms
1000000	42.593614156 s

Cuadro 3: Tiempos de ejecución obtenidos con Go para el Ejercicio 3.

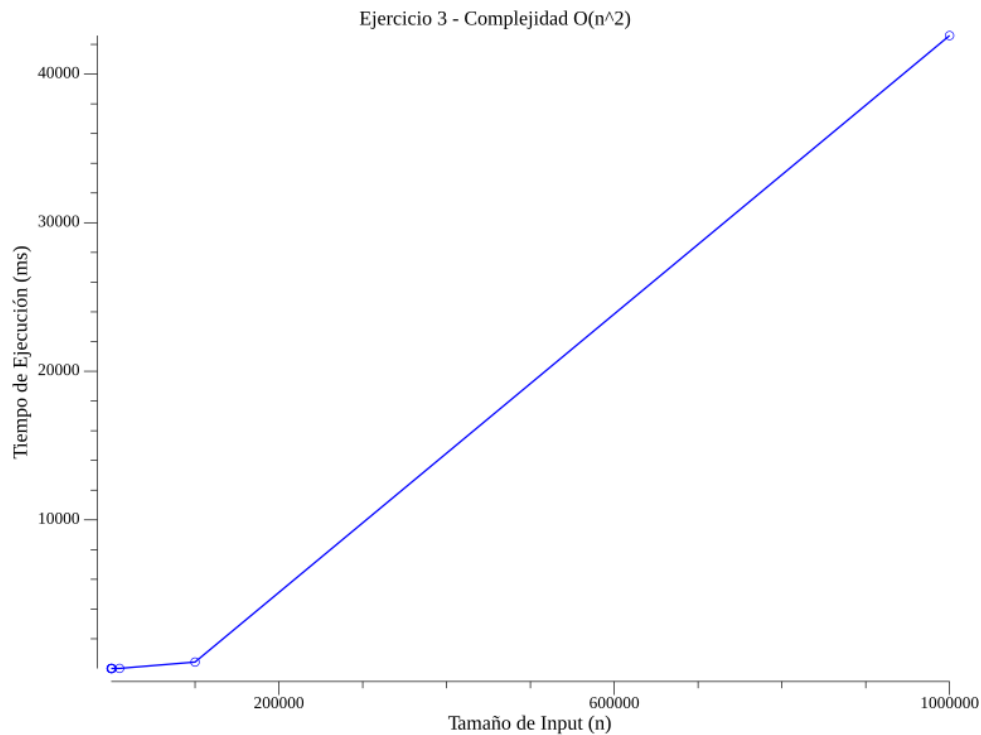


Figura 3: Gráfica de crecimiento cuadrático $O(n^2)$.

Ejercicio 4

Análisis del algoritmo de Búsqueda Lineal (Linear Search) en un arreglo de tamaño n :

- **Mejor caso ($O(1)$):** Ocurre cuando el elemento buscado se encuentra exactamente en la primera posición del arreglo. El algoritmo realiza una única comparación independientemente del tamaño n y termina su ejecución.
- **Peor caso ($O(n)$):** Ocurre cuando el elemento está en la última posición, o cuando simplemente no existe en el arreglo. El algoritmo se ve obligado a recorrer todos los n elementos realizando una comparación por cada uno.
- **Caso promedio ($\Theta(n)$):** Suponiendo que el elemento tiene una distribución probabilística uniforme de encontrarse en cualquier posición i , la probabilidad es $1/n$. El número esperado de comparaciones E se calcula sumando el costo de cada posición ponderado por su probabilidad:

$$E = \sum_{i=1}^n i \cdot \left(\frac{1}{n}\right) = \frac{1}{n} \left(\frac{n(n+1)}{2}\right) = \frac{n+1}{2}$$

Al descartar las constantes asintóticas, la complejidad promedia escala linealmente respecto a n , concluyendo en $\Theta(n)$.

Ejercicio 5

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.

Respuesta: Verdadero. La notación Θ establece una cota ajustada superior e inferior, funcionando matemáticamente como una relación de equivalencia (posee transitividad y simetría). Por transitividad, $f(n) = \Theta(h(n))$. Debido a la simetría, ambas funciones crecen al mismo ritmo, lo que confirma que $h(n) = \Theta(f(n))$.

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.

Respuesta: Verdadero. Por transitividad de la notación O , tenemos que $f(n) = O(h(n))$. Formalmente, esto indica que existe una constante $c > 0$ tal que $f(n) \leq c \cdot h(n)$. Al reestructurar esta desigualdad matemática, obtenemos $h(n) \geq \frac{1}{c} \cdot f(n)$. Esta última expresión define formalmente a $h(n)$ como el límite inferior de $f(n)$, es decir, $h(n) = \Omega(f(n))$.

c) $f(n) = \Theta(n^2)$, donde $f(n)$ es el tiempo de ejecución del programa Python.

Respuesta: Falso. Si bien los dos ciclos `for` anidados realizan aproximadamente $n^2/2$ iteraciones, las operaciones dentro del ciclo interior no son de tiempo constante $O(1)$. La instrucción `atupla[i:j]` en Python crea un nuevo segmento de memoria que requiere un tiempo $O(j-i)$ para copiar los elementos. Adicionalmente, insertar dicho segmento en un conjunto con `S.add()` requiere calcular el hash, tomando un tiempo similar. Sumar este costo lineal adicional a lo largo de las iteraciones cuadráticas eleva la complejidad algorítmica real a $\Theta(n^3)$.